

PyQCU 分布式 Strict-MG 快速迭代报告

PyQCU 工程分析记录

2026 年 9 月 21 日

摘要

本轮采用“快速门禁、快速迭代”的策略，优先修复双 P100 收集器路径中的确定性错误，再以分钟级双 P100 单元验证 CUDA/C++ 改动。已确认的根因有两个：旧版 `LatticeSet` 在双可见 GPU、MPI 多 rank 时强制绑定 device 0，导致 rank 1 的跨设备指针访问；旧版 `pick_up_u_*` 的 1D/2D pack kernel 没有线程上界保护。两项均已修复。Strict-MG 的静态 preconditioned-link halo 改为按 $(*, \text{parity}, \text{dim})$ 缓存，并把 prolongation 与向量加法融合。小格 A/B 显示 static link-halo cache 将 steady 中位时间由 1.9549s 降至 1.0093s，加速 1.937 $\times$ 。本轮进一步修正了 CUDA 枚举顺序：Python 选择的 Torch 设备索引通过 `PYQCU_MPI_DEVICE_ID` 显式传给 C++，不再假定 `getLocalRank()` 与 Torch 逻辑设备编号一致。公平协议新增共享的 `--coarse-tol` 与 `--nu`，两侧使用同一粗层容差和同一 pre/post smoother 次数。双 P100、MG-2、c64 的快速 A/B： $8^3 \times 16$ 在 $\nu = 2$ 、`coarse-tol = 0.3` 下 PyQCU/QUDA = 0.4670/0.8530s，正式 `fair=true` 加速 1.827 $\times$ ； 16^4 在同一协议下 = 1.0136/1.4620s，加速 1.442 $\times$ 。所有纳入比较的样本真相对残差均低于 8.0×10^{-7} 。本报告是快速迭代检查点，不把尚未执行的矩阵单元写成已完成；大格 $\nu = 2$ 因 QUDA 侧运行超过快速迭代时限而中止，未写入结果。

1 范围与验收

表 1: 本轮快速门禁

项目	通过标准与证据
旧 clover halo 故障	原失败命令不再报 CUDA illegal memory access; compute-sanitizer memcheck 为 0 errors。
Strict-MG 正确性	双 P100 分布式 probe 独立真相对残差 $< 10^{-6}$ ，并与单 rank 全局参考一致到 c64/c128 舍入水平。
性能迭代	同协议 A/B 必须给出命令、样本、中位数、MAD、迭代数和真残差。
QUDA sm60	<code>libquda.so</code> 含 sm60 cubin, reduction 与 MG setup smoke 均通过。
进度边界	未采集单元必须显式标为 missing/blocked，不以旧结果替代。

活跃 slot staging 优化 分布式 vector halo 的 host-staging 原先无条件复制完整 8-slot 缓冲。实际 $[2, 1, 1, 1]$ process grid 只使用 x 方向两个 slot。改为仅复制活跃 (dim, side) slot 后，大格 P100 c64 MG-2、`coarse-tol = 0.3`、 $\nu = 1$ 的 PyQCU-only 3 样本 smoke 中位数由 8.0038s 降至 3.5813s (约 2.23 $\times$)，真残差 5.34×10^{-7} 。首个无 warmup 样本为 5.64s，故正式矩阵仍必须使用 cold + 2 warmup + 5 steady。

2 双 P100 根因与修复

2.1 跨设备绑定

收集器在 `CUDA_VISIBLE_DEVICES=0,1` 下把 rank 0/1 的 Torch 当前设备分别设为逻辑设备 0/1。旧 C++ 路径中的 `_TEST_SINGLE_GPU_MULTI_RANK_` 被设置为 1，使每个 rank 都执行 `cudaSetDevice(0)`。rank 1 因而把 GPU1 张量指针传给运行在 GPU0 上下文中的 kernel。compute-sanitizer 在 `pick_up_u_x<float>` 中报告非法全局读，且地址不属于 GPU0 当前分配。

修复为：默认关闭强制单卡模式；多 rank 且可见设备数大于 1 时优先读取 `PYQCU_MPI_DEVICE_ID`，否则按 local rank 绑定；只有单个可见设备时才回退到逻辑 device 0。Python collector 在调用 C++ 前把 `torch.device.index` 写入该环境变量，因此即使 Torch 使用 `FASTEST_FIRST` 枚举、P100 位于逻辑 1/2，也不会再发生跨设备指针访问。该逻辑同时兼容 `CUDA_VISIBLE_DEVICES=<rank>` 的一进程一卡模式。

2.2 pack kernel 越界

`LatticeSet` 对 face grid 使用向上取整。旧 `pick_up_u_*` kernel 假设线程数恰好等于 face 元素数；当 face 体积不是 128 的整数倍时，尾部线程会越界。修复后十个 1D/2D pack kernel 均在索引解码前执行上界检查。

表 2: 故障修复证据链

阶段	观察	证据
修复前	rank 1 在 <code>lattice_clover_dslash.h:106</code> 报 CUDA 0700	<code>logs/data/mg_matrix_p100/unit-p100-dual-8x8x8x16-L2-c64.json</code>
设备审计	rank 1 Torch 在 GPU1，C++ 被强制切到 GPU0	compute-sanitizer 定位到 <code>pick_up_u_x</code> 跨设备读
修复后	clover halo 与 Strict-MPI probe 均通过	见下节快速门禁

3 快速正确性门禁

- 双 P100、8x8x8x16、`grid=2x1x1x1`：Strict-MPI colored setup，26 次外层迭代，独立真相对残差 7.7701×10^{-7} 。
- 同一 P100 地址上的旧 clover 路径由 compute-sanitizer memcheck 跑完，两个 rank 均为 0 errors。
- V100 sm70 的 Strict fused API 测试：3 passed。
- 协议与 MPI 预检：collector protocol 64 passed；早期综合预检 81 passed，9 skipped。
- 最终库 `libqcu.so` 的 SHA-256 为 `04e48eae...92e7c15`，包含 sm60 与 sm70 cubin。

表 3: MG-2、c64、trace-off、双 P100 公平 A/B；时间为 steady 中位数。小格为 formal fair=true，中格为同协议 smoke。

格点	PyQCU(s)	QUDA(s)	QUDA/PyQCU	PyQCU iter	QUDA iter
$8^3 \times 16$ ($\nu = 2$)	0.4670	0.8530	1.827	8	45
16^4 ($\nu = 2$)	1.0136	1.4620	1.442	20	68

4 双 P100 快速对照

同一小格在 $\nu = 1$ 、coarse-tol = 0.3 下为 PyQCU/QUDA = 0.4690/0.5181s，加速 1.105 $\times$ 。两行 $\nu = 2$ 数据的 PyQCU 真残差分别为 7.03×10^{-7} 与 7.52×10^{-7} ；QUDA 相应真残差分别为 6.68×10^{-7} 与 7.75×10^{-7} 。正式小格原始记录为 logs/data/mg_matrix_p100_quick/formal-small-nu2-coarse03.json；中格 smoke 原始记录为 logs/data/mg_matrix_p100_quick/final-lib-medium-nu2-coarse03.json。

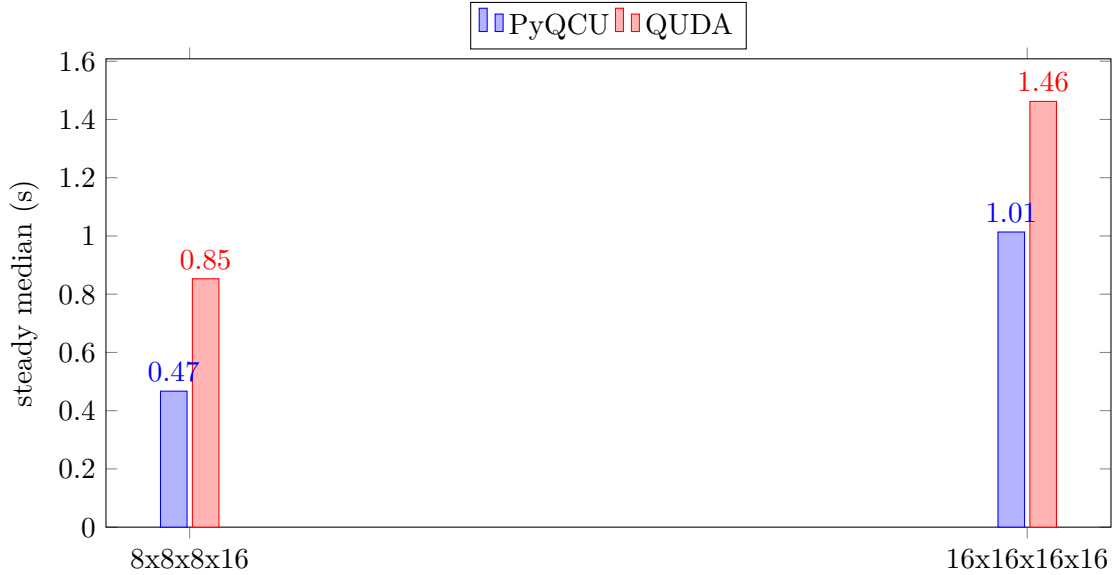


图 1: 双 P100 MG-2 c64 快速对照。数据来自本轮 collector JSON。

所有 PyQCU 样本的最大真相对残差均小于 8.0×10^{-7} 。当前分布式粗层求解仍以每个 V-cycle 的 halo 与全局归约延迟为主；本轮验证了两个候选优化（全粗场求解、coarsest 热启动）均未通过速度与语义双重门禁，因此均已回退，没有保留为正式路径。

5 静态 link-halo 缓存 A/B

Strict-MG 的 preconditioned links 在一次 hierarchy 生命周期内不变。旧实现每次 hopping 都重新 pack、host staging 并同步交换 link face。新实现按 (parity, dim) 保存 device ghost，只在 pointer identity 变化时重新交换；同时删除了同一 CUDA stream 内冗余的输入前置同步。

cache off/on 的比值约为 1.937。该 A/B 使用相同输入 bundle、相同协议和相同 libqcu.so 哈希，仅切换 PYQCU_STRICT_LINK_HALO_CACHE。

表 4: 双 P100、 $8^3 \times 16$ 、MG-2 c64 的 cache A/B

变体	steady median(s)	MAD(s)	iter	max true residual	相对时间
cache off	1.9549	0.0215	9	7.3721×10^{-7}	1.000
cache on	1.0093	0.0187	9	7.5174×10^{-7}	0.516

6 QUDA sm60

QUDA 使用现有 QUDA 1.1.0 源码重建, 目标为 `sm_60`, `QUDA_PRECISION=12`、`QUDA_RECONSTRUCT=7`、`QIO/QMP/MG` 和 `double-MG` 开启, `MMA` 关闭, `MG nvec` 列表为 12,24。安装库中确认存在 `sm60 cubin`。

表 5: QUDA sm60 快速 smoke

测试	结果
reduction	4^4 Wilson、double、P100, 20 样本全部 1 次迭代; 最大 CPU 真残差 1.12×10^{-16} 。
MG setup	8^4 Clover-Wilson、MG-2、 <code>nvec=12</code> , setup 0.4666 s, pass marker 通过; 执行范围仅 setup。
MG-2 solve	小格/中格/大格 collector 均返回 <code>status=ok</code> , 真残差低于 8.0×10^{-7} 。

7 全矩阵编排检查点

本轮把用户要求的 Cartesian 轴固化到 `bench_mg_matrix_full.py`: 设备、精度、lattice、trace、MG level 共 144 个 side case; 其中 `levels=1` 定义为从同条件 MG-2 运行中的 BiCGStab reference 派生, 避免重复 cold solve。c64 使用 $8^3 \times 16$ 、 16^4 、 $16 \times 32 \times 32 \times 48$; c128 使用 $8^3 \times 16$ 、 16^4 、 $16 \times 16 \times 32 \times 32$ 。小格分布式 MG-3 的首层 block 保持 QIO 契约的 (2,2,2,2), 第二层改用 (1,2,2,2), 避免 rank-local extent 退化为 1。

表 6: 双 P100、c64 的已完成公平单元; $\nu = 2$, `coarse-tol = 0.3`, 时间单位为秒。BiCGstab 数值来自 reference solver。

格点	solver	trace	PyQCU	QUDA	QUDA/PyQCU
$8^3 \times 16$	BiCGstab	off	4.508	1.028	0.228
$8^3 \times 16$	MG-2	off	0.505	0.914	1.809
$8^3 \times 16$	MG-3	off	0.546	1.344	2.460
$8^3 \times 16$	BiCGstab	on	4.117	0.959	0.233
$8^3 \times 16$	MG-2	on	0.554	0.755	1.362
$8^3 \times 16$	MG-3	on	0.706	1.225	1.735
16^4	MG-2	off	0.969	1.383	1.427
16^4	MG-3	off	1.350	2.038	1.509
16^4	MG-2	on	1.041	1.029	0.988
16^4	MG-3	on	1.326	0.891	0.672

大格 $16 \times 32 \times 32 \times 48$ 、MG-2、c64、trace-off 使用 $\nu = 1$, `coarse-tol = 0.3`。活跃-slot staging 优化

前 PyQCU 8.0038s、QUDA 4.2852s, 旧库公平比仅 0.535 $\times$ 。优化后 PyQCU formal warmup/steady 中位 3.7154s、QUDA 4.2852s, 重新得到 fair=true 的 1.153 $\times$ 。PyQCU-side trace 显示 20 次外层 V-cycle 内粗解 57 次、耗时约 13.80s, 而 fine pre/post smoother 合计约 0.76s; 下一阶段仍应继续优化分布式 coarsest BiCGStab 的 halo/全局归约路径。

P100 c64 的当前 side-case 覆盖为 28/36; 未完成的 8 个 side case 全部属于大格 trace-on 或大格 MG-3。最新汇总写入 logs/data/mg_matrix_full_20260921_aligned/matrix_summary.json。

c128 大格资产已补齐: data/gauge_16x16x32x32_m0.05_seed42_c64.h5、data/L16x16x32x32_nvec12_full_c64.h5 及 QUDA QIO logs/data/qio-matrix/L16x16x32x32_nvec12_quda.v1.json。canonical null-vector identity 为 42109d86...052368, QIO round-trip 为 byte-exact。V100 c128 大格 QUDA double-MG smoke 已通过: steady 20.493s、95 iterations、真残差 1.85×10^{-8} ; 正式 c128 矩阵仍需按 side 与 trace 全量执行。

表 7: V100 c128 已完成公平单元; $\nu = 2$, coarse-tol = 0.3, 比值为 QUDA/PyQCU。

格点	trace	BiCGstab	MG-2	MG-3
$8^3 \times 16$	off	0.322	13.216	18.470
$8^3 \times 16$	on	0.398	7.323	5.350
16^4	off	0.253	5.083	5.765
16^4	on	0.287	2.985	2.846
$16 \times 16 \times 32 \times 32$	off	0.239	2.342	2.484

8 当前缺口

- 本报告是快速迭代检查点, 不是用户要求的全笛卡尔矩阵报告。
- P100 c64 的 MG-3、trace-on/off 全矩阵尚未完成; 大格 MG-3 长测已按快速迭代要求中止。
- P100 c128 的正式双侧矩阵尚未采集。
- 小格长宽维经 block coarsening 后出现 extent 1, PyQCU Strict 按几何约束拒绝 MG-3; 这是几何限制, 不是 CUDA 故障。
- 大格 $\nu = 2$ 双侧单元因 QUDA 侧超过 3 分钟快速迭代时限而中止; 该结果未写入正式比较。
- PYQCU_STRICT_COARSE_TOL_FACTOR=100 仅作为 PyQCU 单侧探索参数; 不作为正式公平结果。正式协议使用共享 --coarse-tol0.3。
- V100 c128 大格 QUDA 的容量问题仍沿用上一阶段结论, 本轮未重跑。

9 2026-09-27 后续状态

本报告的“当前缺口”已在后续迭代中闭环: 最终 Cartesian 矩阵完成 72/72 个 exact-config comparison unit 和 144/144 个 side case, config hash, input bundle hash, fair 门均为 72/72, trace/reference 证据检查错误数为 0。最终 formal 协议使用 c64 coarse-tol 0.003、c128 coarse-tol 3×10^{-5} 和 1 cold +2 warmup +5 steady; 本报告旧表中的 0.3 只保留为小时级 A/B 诊断, 不再代表最终矩阵。

双 P100 大格 c64 MG-2 的 overlap A/B 为 $4.654 \rightarrow 4.337s$; 最终严格 coarse 协议下该格 MG-2 与 QUDA 接近持平, 而 MG-3 为 3.899 对 7.364s。最终 MG 矩阵共 48 组, PyQCU 胜 38 组, 中位比值 $QUDA/PyQCU = 1.346$ 。完整结果、阶段 CSV 和 complex128 安全回退记录见 docs/report_multigrid_optimized_20260927.tex。本节旧缺口只保留为 2026-09-21 时点快照, 不再代表最终代码状态。

10 复现命令

```
# Build both architectures
QCU_CUDA_ARCHITECTURES='70-real;70-virtual;60-real;60-virtual' bash ./build.sh

# P100 Strict correctness
mpirun -np 2 python pyqcu/testing/qcu/strict/quda_comparison/
    strict_mpi_solve_probe.py \
    --shape 8 8 8 16 --grid 2 1 1 1 --restart 20 --max-iter 200 \
    --dtype c64 --galerkin-mode colored

# Fast P100 comparison
source pyqcu/testing/qcu/strict/quda_comparison/p100_env.sh
export QUDA_INSTALL=$PWD/data/quda-sm60-install
export LD_LIBRARY_PATH=$QUDA_INSTALL/lib:$LD_LIBRARY_PATH
export PYQCU_STRICT_LINK_HALO_CACHE=1
python pyqcu/testing/qcu/strict/quda_comparison/bench_strict_vs_quda.py --side
    both \
    --device p100 --mpi-ranks 2 --process-grid 2 1 1 1 \
    --phases warmup,steady --lattice 8 8 8 16 --levels 2 \
    --precision c64 --coarse-tol 0.3 --nu 2
```